

Microsoft Azure

Build Enterprise & Cloud Applications

Complete Student Notes | From Basics to Advanced Azure Development

Session	Topic
Session 1-2	Cloud Computing Basics & Azure Overview
Session 3-4	Azure Management Tools & Monitoring
Session 5-6	Data Access & Azure Data Services
Session 7-8	Web API Design & RESTful Services
Session 9-10	Azure Security & Hosting Web APIs
Session 11-12	WCF & Serverless / API Management
Session 13	CI/CD with Azure DevOps
Session 14-15	Traffic Management & App Deployment

■ Chapter 0: IT Fundamentals (Before Cloud)

0.1 What is a Network?

A **network** is a collection of computers and devices that are connected to each other so they can share resources and communicate. Think of it like roads connecting cities.

Types of Networks

Type	Full Form	Scope	Real-World Example
LAN	Local Area Network	Single building / office	Office Wi-Fi at a company
WAN	Wide Area Network	Across cities / countries	The Internet itself
MAN	Metropolitan Area Network	Across a city	City-wide cable network
PAN	Personal Area Network	Around a person	Bluetooth headphones ↔ phone

Example: When you connect your laptop to your home Wi-Fi, you join a LAN. When you open Google.com, your request travels over the WAN (Internet) all the way to Google's servers.

Key Network Terms

- **IP Address** – A unique number assigned to every device on a network. Like a home address. Example: 192.168.1.10
- **DNS (Domain Name System)** – Translates human-friendly names (google.com) into IP addresses. Like a phone book for the internet.
- **Router** – A device that forwards data packets between networks. Like a traffic officer on the road.
- **Firewall** – A security system that controls which traffic is allowed in or out of a network.
- **Bandwidth** – The maximum data transfer rate of a network. Like the width of a road — wider = more traffic.
- **Latency** – The delay in data transmission. Lower latency = faster response. Important for cloud apps.

0.2 What is a Server?

A **server** is a powerful computer that provides services (files, websites, databases, emails) to other computers (called **clients**) over a network. A server runs 24/7 and serves many users at the same time.

Example: When you visit a website, your browser (client) sends a request to a web server. The server processes it and sends back the webpage. This is called the **Client-Server Model**.

Types of Servers

- **Web Server** – Serves websites (HTML pages). Example: Apache, IIS, Nginx.
- **Database Server** – Stores and provides data. Example: SQL Server, MySQL.

- **File Server** – Stores and shares files across a network.
- **Mail Server** – Handles emails. Example: Microsoft Exchange.
- **Application Server** – Runs business logic / applications. Example: Azure App Service.

0.3 Shared Hosting vs. Dedicated Hosting vs. Cloud Hosting

Type	Description	Cost	Real Example
Shared Hosting	Multiple websites share the SAME server and its resources. If one site uses too much CPU, others slow down.	Very Cheap (PKR 500–2000/mo)	Hostinger, Namecheap shared plans
VPS (Virtual Private Server)	A physical server is divided into virtual servers. Each VPS acts like its own private server.	Medium (PKR 2000–8000/mo)	DigitalOcean Droplets
Dedicated Hosting	You get an entire physical server for yourself. Full control, high performance.	Expensive (\$100+/month)	Large enterprise websites
Cloud Hosting	Resources are provided on-demand from a pool of servers. Scales automatically. Pay only what you use.	Pay-as-you-go	Microsoft Azure, AWS, Google Cloud

0.4 What is a Domain?

A **domain name** is the human-readable address of a website. For example, **google.com**, **facebook.com**, **microsoft.com**. Instead of remembering an IP address like 142.250.196.46, you just type google.com.

How Domains Work

- Step 1: You type www.google.com in your browser.
- Step 2: Your browser asks a DNS server: 'What is the IP of google.com?'
- Step 3: DNS replies with the IP address (e.g., 142.250.196.46).
- Step 4: Your browser connects to that IP and gets the webpage.

Domain Structure

A domain has parts: **subdomain.domain.extension**

Part	Example	Meaning
Extension (TLD)	.com / .pk / .org / .net	Type of organization or country
Domain Name	google / microsoft / aptech	Unique name of the organization
Subdomain	www / mail / api	Section of the website

Example: In 'mail.google.com': 'mail' is the subdomain, 'google' is the domain, '.com' is the TLD.

0.5 What is the Internet vs. Intranet vs. Extranet?

Term	Who Can Access	Example
Internet	Anyone in the world	Google, YouTube, Facebook
Intranet	Only employees inside a company	Company HR portal, internal email
Extranet	Company + trusted external partners	Bank customer login portal

■ Chapter 1: Introduction to Cloud Computing

1.1 What is Cloud Computing?

Cloud computing is an approach that enables **convenient and on-demand access through the Internet** to computing capabilities and resources. By using cloud computing, services such as databases, software, storage, servers, and networking can be delivered over the Internet — 'the cloud'.

Example: Gmail, Google Drive, Dropbox — you access these from any device without installing anything locally. The data lives on remote servers. This is cloud computing in everyday life.

Tip: Think of cloud computing like electricity: you don't build your own power plant — you just plug in and pay for what you use.

Why Was Cloud Computing Needed?

Imagine a company with 10,000 employees. Their application runs on local servers. Over time, 50,000 more employees join. The servers can't handle the load. Buying new hardware is expensive and slow. Cloud computing solves this — scale up in minutes, pay only for what you use.

Three Core Reasons Organizations Use Cloud

- **Economical:** No upfront hardware purchase. Cloud providers (Amazon, Microsoft) manage infrastructure. You only pay for what you use.
- **Scalability:** Scale up or down in minutes. If your app needs more storage, add it instantly without downtime.
- **Deployment:** Cloud environments match production environments, preventing 'it works on my machine' failures.

1.2 Benefits of Cloud Computing

Benefit	Explanation	Example
Infinite Resources	No planning for storage — the cloud scales for you.	Netflix stores petabytes of video on AWS
Scale on Demand	Add resources exactly when needed, remove when not needed.	E-commerce site adds servers on sales day
Pay-for-use (OpEx)	No capital expenditure. Pay monthly operating costs only.	Pay only for VMs you actually run
High Availability	SLAs guarantee uptime (Azure: 99.9%–99.99%). Automatic failover.	Your app stays online even if one datacenter fails
Global Datacenters	Deploy close to your users for low latency.	A Karachi user gets served from a nearby Azure region

Example: Capital Expenditure (CapEx) = Buying a server upfront for Rs. 500,000. Operating Expenditure (OpEx) = Paying Azure Rs. 5,000/month for a virtual server. Cloud converts CapEx to OpEx.

1.3 Cloud Service Models

There are three main cloud service models. Think of it as how much control you keep vs. how much the provider manages:

Model	Full Name	What Cloud Provides	What You Manage	Real Examples
IaaS	Infrastructure as a Service	Servers, storage, networking, VMs, OS	Applications, runtime, data	Azure VMs, AWS EC2
PaaS	Platform as a Service	Everything in IaaS + OS, runtime, middleware	Only your application & data	Azure App Service, Azure SQL
SaaS	Software as a Service	Everything — complete application	Just your data and users	Gmail, Office 365, Dropbox

Example: IaaS = Renting a raw plot of land. PaaS = Renting a furnished apartment. SaaS = Staying in a hotel (fully managed, you just live there).

1.4 Cloud Deployment Models

Model	Description	Best For
Public Cloud	Resources owned and operated by a cloud provider (Azure, AWS, GCP). Shared infrastructure.	Startups, general applications
Private Cloud	Cloud infrastructure operated solely for a single organization. On-premises or hosted.	Government, banking, sensitive data
Hybrid Cloud	Combination of public and private clouds connected together.	Enterprises needing both flexibility and security
Community Cloud	Shared by organizations with common concerns (e.g., government agencies).	Regulatory compliance sectors

■ Chapter 2: Understanding Microsoft Azure

2.1 What is Microsoft Azure?

Microsoft Azure is a **cloud computing platform** used for building, deploying, and managing services and applications. It uses a global network of Microsoft-managed data centers. Azure supports both **IaaS** and **PaaS** services and works with Microsoft and third-party technologies.

Azure's Three Core Actions

- **Build:** Create software applications, write code, design user interfaces, develop functionality.
- **Deploy:** Make the software available — configure servers, set up databases, ensure resources are ready.
- **Manage:** Monitor performance, maintain uptime, troubleshoot, scale resources, ensure security.

2.2 Azure Platform Components

Category	Key Services	Purpose
Compute	Virtual Machines, Container Registry, Azure Functions, Batch, Service Fabric	Run applications and process data
Storage	Blob, Table, Queue, File, Disk Storage	Store all types of data at scale
Networking	Virtual Network, Load Balancer, Application Gateway, Azure DNS, Traffic Manager	Connect resources and control traffic
Databases	Azure SQL, Cosmos DB, Redis Cache, Data Warehouse	Managed database services
App Services	Web Apps, Mobile Apps, API Apps	Host and manage web/mobile/API apps
Security	Key Vault, Microsoft Entra ID, Azure AD B2C	Protect data and manage identities
AI/Cognitive	Cognitive Services APIs	Add AI features (vision, speech, NLP) to apps
Analytics	Azure Data Lake, Stream Analytics, Databricks, Synapse	Process and analyze big data
IoT	IoT Hub, IoT Central	Connect and manage IoT devices
DevOps	Azure DevOps, Azure Pipelines	CI/CD, testing, deployment automation

2.3 Azure Compute Services (Detailed)

Virtual Machines (VMs)

A **Virtual Machine (VM)** is a software emulation of a computer. Using a **hypervisor**, multiple VMs run on one physical server. Each VM has its own OS, CPU, memory, and network interface.

Example: Instead of buying a physical Windows Server for Rs 500,000, you create an Azure VM in 5 minutes and pay Rs 2,000/month. Delete it when you don't need it.

- Supports Windows and Linux operating systems
- Full control over OS, software installed, configuration
- Virtual Machine Scale Sets: create thousands of identical VMs in seconds for horizontal scaling
- Use case: hosting web applications, running databases, development/testing environments

Containers

A **container** is a lightweight package that contains everything needed to run an application: code, runtime, libraries, config. Unlike VMs, containers share the host OS — making them **faster and smaller**.

Feature	Virtual Machine	Container
Size	GBs (includes full OS)	MBs (shares host OS)
Startup Time	Minutes	Seconds
Isolation	Full hardware isolation	Process-level isolation
Portability	Less portable	Highly portable
Use case	Full OS required	Microservices, CI/CD

Example: Docker is a popular container tool. Azure Container Instances (ACI) lets you run containers without managing servers. Azure Kubernetes Service (AKS) orchestrates many containers together.

Azure Functions (Serverless)

With **Azure Functions**, you write code that runs in response to events — without managing any server. Azure handles all the infrastructure automatically. This is called **Serverless Computing**.

Example: You write a function: 'When a new image is uploaded to Blob Storage → resize it.' The function runs only when triggered. You pay only for execution time, not idle time.

2.4 Azure Storage Services

Service	Type	Use Case	Example
Blob Storage	Object/Unstructured data	Store images, videos, backups, log files	Storing profile pictures for a social media app
Table Storage	NoSQL Key-Value	Semi-structured data, no complex joins needed	Storing device sensor data or email addresses

Service	Type	Use Case	Example
Queue Storage	Message queue	Async communication between app components	Order processing system — orders wait in queue
File Storage	SMB file share	Legacy app migration, shared file access	Replacing on-premises file servers
Disk Storage	Block storage for VMs	OS disks and data disks for Virtual Machines	Hard drive for an Azure VM

Blob Types

Blob Type	Description	Max Size	Use Case
Block Blob	Stores text and binary data in blocks	50,000 blocks × 100 MB each	Videos, images, documents
Append Blob	Optimized for append operations	50,000 blocks × 4 MB each	Log files
Page Blob	Random read/write access	Up to 8 TB	OS disks, virtual hard disks

2.5 Azure Networking Services

Virtual Network (VNet)

Creates an isolated private network in Azure. You control IP ranges, subnets, routing, and security.

Example: Your company's Azure resources talk privately without going through the internet.

Load Balancer

Distributes incoming traffic across multiple VMs/servers so no single server is overloaded.

Example: If 1000 users hit your website, load balancer sends 250 each to 4 VMs.

Application Gateway

Advanced web traffic manager that works at Layer 7 (HTTP/HTTPS). Includes Web Application Firewall (WAF).

Example: Route /api/* requests to API servers, /images/* to storage.

Azure DNS

Hosts your domain names in Azure and provides fast, reliable name resolution.

Example: yourcompany.com → resolves to Azure IP using Azure's global DNS servers.

Traffic Manager

DNS-based load balancer that routes users to the nearest/healthiest endpoint globally.

Example: Pakistani users go to Middle East Azure region; European users go to Europe region.

VPN Gateway

Connects your on-premises network to Azure over an encrypted VPN tunnel.

Example: Your company office in Karachi securely connects to Azure resources as if they're on the same network.

2.6 Azure Database Services

Service	Type	Best For	Key Feature
Azure SQL Database	Relational (SQL)	Traditional apps needing SQL	Fully managed SQL Server in the cloud. Auto-backups, high availability.
Azure Cosmos DB	NoSQL (multi-model)	Global, low-latency apps	Data replicated globally. Configurable consistency levels.
Azure Redis Cache	In-memory cache	Speeding up frequently accessed data	Sub-millisecond response time. Reduces database load.
Azure SQL Data Warehouse (Synapse)	Big data analytics	Large-scale data analysis	Massively Parallel Processing (MPP) for fast analytics on huge datasets.
Azure Cosmos DB for MongoDB	NoSQL Document DB	Apps already using MongoDB	Full MongoDB API compatibility on Azure's global infrastructure.

Azure SQL Deployment Options

Option	Description	Use Case
Single Database	One isolated database with its own resources on a logical server.	Apps with predictable, steady workloads
Elastic Pool	Multiple databases sharing a common set of resources. Cost-efficient.	Multiple apps with variable/unpredictable load patterns
Managed Instance	Near 100% SQL Server compatibility. Best for migrating from on-premises.	Lift-and-shift migration of existing SQL Server apps

Azure Cosmos DB Features

- **Throughput:** Measured in Request Units (RUs). Scale up/down as needed.
- **Low Latency:** Data replicated globally — users always read from nearby datacenter. Single-digit ms reads.
- **High Availability:** Data replicated across multiple datacenters. Auto-failover if one goes down.

- **Multiple APIs:** Supports SQL API, MongoDB API, Cassandra API, Gremlin API, Table API.
- **Consistency Levels:** Choose from 5 levels: Strong, Bounded Staleness, Session, Consistent Prefix, Eventual.

2.7 Azure App Services

Azure App Service is the **fastest way to publish web projects**. It is a fully managed PaaS — you don't worry about servers, OS, patches, or infrastructure.

App Service Type	Purpose	Languages Supported
Web Apps	Host web applications and APIs	Python, Java, Node.js, PHP, .NET, Ruby
Mobile Apps	Backend for mobile applications	.NET, Node.js, Java
API Apps	Host and expose APIs securely with Swagger support	Any language via REST

Example: You build a .NET web app in Visual Studio → right-click → Publish → Azure Web App. In minutes your app is live at yourapp.azurewebsites.net with SSL, auto-scaling, and monitoring included.

2.8 Azure Advantages

- **Pricing Flexibility:** Pay-per-use. Storage, compute, bandwidth billed separately. No upfront cost.
- **Scalability:** Scale up/down in minutes. No need to buy new hardware.
- **High Availability:** 99.95% uptime SLA guaranteed. Money-back if Azure fails to meet it.
- **Effective Management:** Azure Fabric Controller auto-manages OS updates, patches, instance recovery.
- **Seamless Integration:** Azure Connect bridges on-premises infrastructure with Azure apps.
- **Global Reach:** 60+ Azure regions worldwide. Deploy close to your users for low latency.
- **Security:** Built-in security features — encryption, identity management, threat detection.

Service Level Agreement (SLA)

An **SLA** is a formal commitment Microsoft makes about uptime. For example:

Azure Service	SLA Uptime	Monthly Downtime Allowed
Azure Virtual Machines (2+ instances)	99.99%	~4.3 minutes
Azure SQL Database	99.99%	~4.3 minutes
Azure Blob Storage	99.9%	~43.8 minutes
Azure App Service	99.95%	~21.9 minutes

■ Chapter 3: Azure Management Tools

3.1 Azure CLI (Command-Line Interface)

The Azure CLI is a **cross-platform command-line tool** for controlling Azure resources. It works on Windows, macOS, Linux, and in Azure Cloud Shell directly in the browser.

Example: Create a VM with one command: `az vm create --resource-group MyRG --name MyVM --image UbuntuLTS --admin-username azureuser --generate-ssh-keys` List all VMs: `az vm list --output table`

Key Azure CLI Features

- Cross-platform: Works on Linux, Mac, Windows, and Docker containers
- Multi-Shell: Works in PowerShell, Bash, Windows Command Prompt
- Automation: Perfect for scripting repetitive tasks (create 50 VMs with a loop)
- Interactive Mode: ``az interactive`` provides auto-complete and help text
- Output Formats: JSON, YAML, table, TSV — choose what suits your script

3.2 Azure PowerShell

Azure PowerShell is a set of **cmdlets (commands)** that let you manage Azure from PowerShell. Best for Windows administrators who are already familiar with PowerShell scripting.

Example: Get all VMs: `Get-AzVM` Create a resource group: `New-AzResourceGroup -Name 'MyRG' -Location 'EastUS'` Start a VM: `Start-AzVM -ResourceGroupName 'MyRG' -Name 'MyVM'`

***Tip:** Use Azure CLI if you prefer Linux/Bash. Use Azure PowerShell if you prefer Windows PowerShell. Both do the same things — choose what you're comfortable with.*

3.3 Azure Cloud Shell

Azure Cloud Shell is a **browser-based shell** accessible directly from the Azure Portal. No installation needed. Supports both Bash (Azure CLI) and PowerShell. Comes with pre-installed tools: git, terraform, kubectl, helm, and more.

Example: Go to portal.azure.com → click the '>_' icon in the top bar → Cloud Shell opens. You can run az and PowerShell commands immediately without installing anything on your computer.

3.4 Azure Resource Manager (ARM)

ARM is the **deployment and management service** for Azure. All tools (Portal, CLI, PowerShell, REST APIs) go through ARM. It provides consistent management, role-based access control (RBAC), and tagging across resources.

ARM Templates

ARM Templates are **JSON files** that describe the infrastructure you want to deploy. Infrastructure as Code (IaC) — define once, deploy many times consistently.

Example: An ARM template to create a storage account: { "type": "Microsoft.Storage/storageAccounts", "name": "mystorageaccount", "location": "eastus", "kind": "StorageV2", "sku": { "name": "Standard_LRS" } }
Run: az deployment group create --template-file storage.json --resource-group MyRG

3.5 Azure Portal

The Azure Portal (portal.azure.com) is the **web-based graphical interface** for managing all Azure resources. It includes Dashboards, Resource Groups, all Azure services, and monitoring tools.

Azure Dashboard

Dashboards provide a focused, customizable view of your most important resources. You can create multiple dashboards, pin tiles (charts, metrics, resource shortcuts), and share dashboards with your team.

3.6 Azure Resource Groups

A **Resource Group** is a logical container that holds related Azure resources (VMs, databases, storage accounts, etc.) for a solution. All resources in a group share the same lifecycle — deploy, update, delete together.

Example: You create a Resource Group called 'EcommerceApp' containing: 1 Azure SQL Database, 2 Azure VMs (web servers), 1 Load Balancer, 1 Storage Account. When you're done, delete the group → everything inside is deleted automatically.

■ Chapter 4: Azure Monitoring

4.1 Why Monitor?

Monitoring ensures your applications and infrastructure run optimally, securely, and reliably. It helps detect problems before users notice them.

4.2 Azure Monitor

Azure Monitor is the **central monitoring platform** for all Azure resources. It collects, analyzes, and responds to telemetry (metrics, logs, traces) from cloud and on-premises environments.

What Azure Monitor Can Do

Capability	Description
Performance Management	Collect CPU, memory, disk, network metrics from VMs, databases, apps
Proactive Issue Detection	Set alerts that fire before problems impact users
Troubleshooting	Analyze logs to find root cause of issues
Optimization	Identify underutilized resources to save cost
Security & Compliance	Detect anomalies, unauthorized access, audit trails
Cost Management	Monitor resource usage to optimize spending

Types of Metrics

Metric Type	Description	Example
Platform Metrics	Automatically collected by Azure for its services	VM CPU usage, Storage transactions/second
Custom Metrics	User-defined metrics from your application code	Number of orders placed per minute in your app
Prometheus Metrics	From Kubernetes clusters (AKS), analyzed with PromQL/Grafana	Pod CPU usage in a Kubernetes cluster

4.3 Log Analytics & KQL

Log Analytics stores and queries log data using **Kusto Query Language (KQL)** — a powerful language designed for log data analysis.

Example: Sample KQL Query — Find errors in the last hour: AppRequests | where TimeGenerated > ago(1h) | where Success == false | summarize count() by bin(TimeGenerated, 5m) | render timechart This finds all failed requests in the last hour, groups them by 5-minute intervals, and shows a time chart.

4.4 Azure Application Insights

Application Insights is an **Application Performance Management (APM)** service. It monitors live web applications, detects performance anomalies, and provides analytics.

What Application Insights Monitors

- Request rates, response times, failure rates
- Code-level performance: slow database queries, slow dependencies
- Availability tests: check if your app is reachable every X minutes
- User behavior: which pages users visit, how long they stay
- Custom events: track specific actions in your application
- Real-time metrics stream: see live traffic on your app

Example: You add Application Insights SDK to your ASP.NET app. Now you can see: 'Average response time: 230ms, 99th percentile: 1.2s, Error rate: 0.3%'. You see that /checkout endpoint is slow — drill down and find it's a slow SQL query.

4.5 Azure Alerts

Alerts notify you when specific conditions are met in your monitoring data.

Alert Type	Based On	Example
Metric-Based Alert	Numeric threshold on a metric	CPU > 90% for more than 5 minutes
Log-Based Alert (KQL)	KQL query result	More than 10 errors in the last 10 minutes
Activity Log Alert	Azure resource changes	Alert when someone deletes a VM
Smart Detection	Machine learning anomaly detection	Response time suddenly 5x slower than normal
Prometheus Alert	PromQL query on Kubernetes metrics	Pod memory usage > 80%

Alert Conditions

Concept	Description
Threshold	The value that triggers the alert. Example: CPU > 80%
Time Window	How long condition must persist. Example: CPU > 80% for 5 min (prevents false alerts from brief spikes)
Action Group	What to do when alert fires: email, SMS, run Azure Function, call webhook

4.6 Infrastructure Monitoring

Component	What is Monitored
Virtual Machines	CPU, disk I/O, memory, network traffic. VM Insights for deeper analysis.
Azure SQL / Cosmos DB	Query performance, DTU/RU usage, connections, data storage
Storage Accounts	Throughput, latency, capacity utilization, transaction errors
Virtual Networks	Traffic flows, connectivity, latency, packet loss via Azure Network Watcher
Azure Resource Health	Real-time status of all Azure services — is the problem your config or Azure itself?

■ Chapter 5: Data Access with Entity Framework

5.1 What is Entity Framework (EF)?

Entity Framework is an **Object-Relational Mapper (ORM)** framework developed by Microsoft for .NET applications. It lets developers work with databases using **.NET objects and classes** instead of writing raw SQL queries.

Example: Without EF: Write SQL — 'SELECT * FROM Students WHERE StudentName = "Ali"'. With EF: Write C# — `var student = context.Students.Where(s => s.StudentName == "Ali").FirstOrDefault();` EF translates this to SQL automatically.

Key Features of Entity Framework

Feature	Description
ORM	Maps database tables to .NET classes automatically. No manual SQL needed.
Database Independence	Works with SQL Server, MySQL, PostgreSQL, SQLite, Oracle. Change database with minimal code changes.
LINQ Integration	Write type-safe queries in C# using LINQ. IntelliSense helps avoid errors.
Change Tracking	EF automatically tracks which objects changed and generates UPDATE/INSERT/DELETE SQL.
Code-First	Define your C# classes, EF creates the database schema.
Database-First	Start with existing database, EF generates C# classes.
Migrations	Evolve database schema over time with migration files — version control for your database.
Lazy/Eager Loading	Control when related data is loaded from the database.

5.2 EF Design Approaches

Approach	You Start With	EF Creates	Best For
Code-First	C# classes (POCO entities)	Database schema (tables, keys, relationships)	New projects, Domain-Driven Design
Database-First	Existing database with tables	C# model classes and DbContext	Legacy databases, existing systems
Model-First	Visual model in EF Designer (EDMX)	Both database schema and C# classes	Design-first approach with visual modeling

Example: CODE-FIRST EXAMPLE: public class Student { public int StudentId { get; set; } public string StudentName { get; set; } public string Email { get; set; } } public class SchoolContext : DbContext { public DbSet Students { get; set; } } // Run: Add-Migration InitialCreate // Run: Update-Database // EF creates the 'Students' table automatically!

5.3 LINQ Queries with Entity Framework

LINQ (Language-Integrated Query) is a .NET feature that allows you to write database queries in C# with full IntelliSense and type safety.

Common LINQ Operations

Operation	LINQ Code	SQL Equivalent
Filter (WHERE)	.Where(s => s.Age > 18)	WHERE Age > 18
Sort (ORDER BY)	.OrderBy(s => s.Name)	ORDER BY Name
Select columns (SELECT)	.Select(s => new { s.Name, s.Age })	SELECT Name, Age
Count	.Count()	SELECT COUNT(*)
First match	.FirstOrDefault()	SELECT TOP 1 ...
Join	.Join(other, s => s.Id, o => o.StudentId, ...)	INNER JOIN
Group By	.GroupBy(s => s.Department)	GROUP BY Department
Aggregate	.Sum(s => s.Score) / .Average(s => s.Score)	SUM() / AVG()

Example: LINQ Query Examples: // Get all students with grade A var topStudents = context.Students .Where(s => s.Grade == "A") .OrderBy(s => s.Name) .ToList(); // Count students per department var deptCounts = context.Students .GroupBy(s => s.Department) .Select(g => new { Dept = g.Key, Count = g.Count() }) .ToList();

5.4 Eager Loading vs. Lazy Loading

Loading Type	Description	Code	Best When
Eager Loading	Related data loaded together in the SAME query using Include()	.Include(s => s.Address)	You KNOW you'll need the related data (avoids N+1 problem)
Lazy Loading	Related data loaded ONLY when accessed for the first time. Requires virtual property.	var addr = student.Address; // loaded on demand	You MIGHT need related data but aren't sure
Explicit Loading	Manually load related data when needed using Load()	context.Entry(s).Reference(s => s.Address).Load()	Fine-grained control over loading

Example: N+1 Problem (bad): If you have 100 students and load their addresses with lazy loading, EF runs 1 query for students + 100 queries for addresses = 101 queries. With Eager Loading (`.Include()`), it runs only 1 query with a JOIN. Always use Eager Loading when you know you'll need related data.

■ Chapter 6: Working with Azure Data Services

6.1 Data Ingestion into Azure

Data ingestion is the process of **transferring data from various sources into Azure** storage or data services for analysis, processing, or storage.

Ingestion Process Steps

- **Step 1 — Source Identification:** Identify where data comes from: databases, files, IoT devices, APIs, streaming platforms
- **Step 2 — Data Preparation:** Clean, format, and transform data. Resolve schema differences.
- **Step 3 — Choose Ingestion Method:** Based on volume, velocity, variety, and latency requirements.
- **Step 4 — Ingest the Data:** Use the selected tool/service to move the data.
- **Step 5 — Validate & Monitor:** Verify accuracy and completeness. Monitor for errors.
- **Step 6 — Store & Access:** Place data in the right Azure service for its use case.

Azure Ingestion Tools

Tool	Use Case	Example
Azure Data Factory	Orchestrate ETL workflows: move & transform data at scale	Move data from on-premises SQL Server to Azure SQL Database nightly
Azure Event Hubs	Real-time streaming data ingestion for millions of events per second	Collect click events from a website in real-time
Azure IoT Hub	Ingest data from IoT devices securely	Temperature sensors sending readings every second
Azure Blob Storage	Direct file upload from on-premises or cloud	Upload CSV files daily for batch processing
Azure Data Box	Physical device for transferring massive amounts of data (PBs)	Moving 100TB of data from a datacenter to Azure

6.2 Azure Data Services Overview

Service	Type	Best For
Azure SQL Database	Relational PaaS	Traditional transactional apps
Azure Cosmos DB	NoSQL multi-model	Global, low-latency, flexible schema apps
Azure Data Lake Storage (ADLS)	Big data storage	Analytics on huge datasets (Hadoop-compatible)

Service	Type	Best For
Azure Synapse Analytics	Analytics platform	Combining big data + data warehouse + analytics
Azure Databricks	Apache Spark platform	Machine learning, data science, real-time analytics
Azure Stream Analytics	Real-time stream processing	IoT, fraud detection, live dashboards

6.3 Caching in Azure

Caching stores frequently accessed data locally (in memory) so future requests don't need to hit the database, improving performance significantly.

Types of Caching

- **Static Content Caching:** Cache images, CSS, JavaScript files. These don't change often. Store on CDN edge servers close to users.
- **Redis Cache (Azure Cache for Redis):** In-memory key-value store. Sub-millisecond response. Cache database query results, session data.
- **Response Caching:** Cache full HTTP responses. If request is identical, return cached response without hitting the server.
- **CDN (Content Delivery Network):** Distribute static content globally at edge servers. Reduces latency for geographically distributed users.

Example: Without Redis Cache: Every user request hits the database → 200ms response time. With Redis Cache: User request hits cache first → if found (cache hit) → 2ms response. Only if not in cache (cache miss) → hit database → store in cache for next time.

6.4 Transactions & ACID Properties

A **transaction** is a single unit of work — either ALL operations succeed, or NONE of them apply. This maintains data consistency.

Example: Bank Transfer: Step 1 — Deduct Rs 1000 from Account A. Step 2 — Add Rs 1000 to Account B. If Step 2 fails, Step 1 MUST be rolled back. The transfer never partially happens. This is a transaction.

ACID Properties

Property	Meaning	Bank Transfer Example
Atomicity	All or nothing. Either all operations in the transaction succeed, or none.	Both deduct and credit happen, or neither.
Consistency	Database moves from one valid state to another. Rules and constraints maintained.	Total money in bank stays the same before and after.
Isolation	Concurrent transactions don't interfere. Each sees consistent data.	Another user's transfer doesn't see your partial transfer.

Property	Meaning	Bank Transfer Example
Durability	Once committed, changes are permanent even if system crashes.	Transfer is permanently recorded even if server crashes right after.

Transaction Isolation Levels

Level	Allows Dirty Read	Allows Non-Repeatable Read	Allows Phantom Read	Use Case
Read Uncommitted	YES	YES	YES	Reports that can tolerate approximate data
Read Committed	NO	YES	YES	Default for most databases (SQL Server)
Repeatable Read	NO	NO	YES	When you need stable reads within a transaction
Serializable	NO	NO	NO	Highest integrity, lowest concurrency. Financial systems.

6.5 Content Delivery Network (CDN)

A **CDN** is a distributed network of servers (called Points of Presence / POPs) placed around the world. Static content is cached at POPs near users, reducing latency dramatically.

Example: Your website server is in East US. A user in Karachi downloads a 10MB image. Without CDN: downloads from US server → 300ms. With CDN: downloads from Dubai POP (nearest) → 20ms. Azure CDN caches your content at 200+ locations worldwide.

CDN Benefits

- Reduces latency — users get content from nearby server
- Reduces origin server load — cache serves most requests
- Handles traffic spikes — product launch, viral content
- SSL/TLS encryption at the edge
- DDoS protection at edge locations

■ Chapter 7: Design & Implementation of Web API

7.1 What is a Web API?

A **Web API (Application Programming Interface)** is a set of endpoints that allows different applications to communicate over HTTP. ASP.NET Web API is Microsoft's framework for building HTTP services accessible to browsers, mobile apps, and other clients.

Example: A mobile banking app needs to check your balance. Instead of giving the app direct database access, the bank exposes a Web API: `GET /api/accounts/{id}/balance` → returns `{balance: 50000}`. The app calls this API — secure, controlled, reusable.

Why Use a Centralized Web API?

Without a central API: Each mobile app (Android, iOS, Web) has its own business logic and database connection. Any change requires updating ALL apps. With a central API: One API serves all clients. Update business logic once. All apps automatically get the update.

7.2 HTTP Methods (REST Verbs)

HTTP Method	Operation	Example URL	Description
GET	READ	GET /api/students	Retrieve list of students
GET	READ	GET /api/students/5	Get student with ID 5
POST	CREATE	POST /api/students	Create a new student (body contains data)
PUT	UPDATE (full)	PUT /api/students/5	Replace student 5 with new data
PATCH	UPDATE (partial)	PATCH /api/students/5	Update only specific fields of student 5
DELETE	DELETE	DELETE /api/students/5	Delete student with ID 5

7.3 HTTP Request & Response Structure

HTTP Request Headers

Header	Purpose	Example Value
Host	Domain name of the server	api.myapp.com
Accept	What format the client expects (MIME type)	application/json

Header	Purpose	Example Value
Authorization	Authentication token	Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type	Format of data in request body	application/json
Accept-Language	Preferred language	en-US, ur-PK

HTTP Response Status Codes

Code Range	Meaning	Common Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

7.4 Routing in Web API

Convention-Based Routing (Default)

Web API maps URLs to controller actions using a routing table. The default template is: **api/{controller}/{id}**

Example: // WebApiConfig.cs config.Routes.MapHttpRoute(name: "DefaultApi", routeTemplate: "api/{controller}/{id}", defaults: new { id = RouteParameter.Optional }); // Requests: // GET /api/students → StudentsController.GetAll() // GET /api/students/5 → StudentsController.Get(5) // POST /api/students → StudentsController.Post(student) // DELETE /api/students/5 → StudentsController.Delete(5)

Attribute Routing

Attribute routing puts route definitions directly on the controller method using [Route] attribute. Gives more control over URL structure.

Example: [RoutePrefix("api/students")] public class StudentsController : ApiController { [HttpGet, Route("")] // GET /api/students public IEnumerable GetAll() { ... } [HttpGet, Route("{id:int}")] // GET /api/students/5 public Student Get(int id) { ... } [HttpGet, Route("{id}/courses")] // GET /api/students/5/courses public IEnumerable GetCourses(int id) { ... } }

7.5 Authentication in Web API

Method	Description	Use Case
Basic Auth	Username+password sent with every request (Base64 encoded). Simple but less secure.	Simple internal APIs over HTTPS

Method	Description	Use Case
Token-Based (JWT)	Client logs in once, gets a JWT token. Sends token in every request header. Server validates token.	Modern web/mobile APIs. Stateless.
OAuth 2.0	Industry standard. Client gets access token from authorization server.	Third-party access (Login with Google/Facebook)
API Key	A unique key per client, sent in header or URL.	Public APIs, machine-to-machine communication

Example: JWT Token Flow: 1. Client: POST /api/auth/login {username:"ali", password:"pass123"} 2. Server: validates → returns JWT token: "eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJhbGkiLCJleHAiOiJlE3..." 3. Client: stores token, sends with every request: GET /api/students Authorization: Bearer eyJhbGciOiJIUzI1NiJ9... 4. Server: validates token signature and expiry → allows/rejects

7.6 CORS & Security

CORS (Cross-Origin Resource Sharing)

By default, browsers block JavaScript from making requests to a different domain (Same-Origin Policy). CORS allows servers to explicitly permit cross-origin requests.

Example: Your frontend is on myapp.com. Your API is on api.myapp.com. Without CORS: browser blocks the request. With CORS enabled on API: browser allows it. Enable in Web API: `config.EnableCors(new EnableCorsAttribute('https://myapp.com', '*', '*'));`

XSRF / CSRF (Cross-Site Request Forgery)

A CSRF attack tricks an authenticated user's browser into making unwanted requests to a website. Example: you're logged into your bank. A malicious site loads in another tab and sends a hidden request to transfer money from your account.

Tip: Prevent CSRF: Use anti-forgery tokens (hidden form fields that prove the request came from your own page). ASP.NET has built-in `[ValidateAntiForgeryToken]` attribute for MVC/Web API.

■ Chapter 8: RESTful APIs for Enterprise Applications

8.1 What is REST?

REST (Representational State Transfer) is an architectural style for designing web APIs. A RESTful API uses standard HTTP methods (GET, POST, PUT, DELETE) and treats everything as a **resource** with a URL.

REST Principles

Principle	Description
Stateless	Each request contains all info needed. Server doesn't store session state.
Client-Server	Frontend and backend are separated. They communicate via API.
Cacheable	Responses can be cached. Reduces server load.
Uniform Interface	Consistent URL structure and HTTP verbs. Resources identified by URLs.
Layered System	Client doesn't know if it talks to actual server or intermediary (load balancer, cache).

REST vs. SOAP

Feature	REST	SOAP
Protocol	HTTP	Any (HTTP, SMTP, TCP)
Data Format	JSON (usually), XML	XML only
Complexity	Simple, lightweight	Complex, verbose
Performance	Faster (smaller payloads)	Slower (XML overhead)
Security	HTTPS + OAuth/JWT	WS-Security (more built-in options)
Use Case	Web/mobile apps, microservices	Enterprise systems, banking, legacy apps

8.2 OData Services

OData (Open Data Protocol) is a standard for building and consuming RESTful APIs with powerful querying capabilities built into the URL.

Example: OData URL Examples: GET /odata/Students → All students GET /odata/Students?\$top=10 → First 10 students GET /odata/Students?\$filter=Age gt 18 → Students older than 18 GET /odata/Students?\$orderby=Name asc → Sorted by name GET /odata/Students?\$select=Name,Email → Only Name and Email fields GET /odata/Students?\$expand=Courses → Include related courses GET /odata/Students/\$count → Total count of students Combine: /odata/Students?\$filter=Grade eq 'A'&\$orderby=Name&\$top=5

8.3 Dependency Injection in Web API

Dependency Injection (DI) is a design pattern where a class receives its dependencies from outside rather than creating them internally. This makes code more testable, maintainable, and loosely coupled.

Example: WITHOUT DI (tightly coupled): `public class StudentsController : ApiController { private StudentRepository repo = new StudentRepository(); // Hard-coded dependency }` WITH DI (loosely coupled): `public class StudentsController : ApiController { private IStudentRepository _repo; // Depends on interface public StudentsController(IStudentRepository repo) { // Injected by container _repo = repo; } } // Register in DI container: container.Register();`

Tip: With DI, you can swap `StudentRepository` with `MockStudentRepository` during unit testing without changing controller code. This is testability in action.

■ Chapter 9: Azure Security Features

9.1 Azure Security Overview

Azure provides a comprehensive set of security tools across all layers: identity, network, data, and applications.

9.2 Microsoft Entra ID (formerly Azure Active Directory)

Entra ID is Azure's **cloud-based identity and access management service**. It authenticates users and authorizes access to resources.

Key Features

- **Single Sign-On (SSO):** Log in once, access multiple apps (Office 365, Salesforce, GitHub, etc.)
- **Multi-Factor Authentication (MFA):** Require a second factor (phone app, SMS) beyond password.
- **Conditional Access:** Allow/block access based on user location, device, risk level.
- **Device Registration:** Register company devices for additional security policies.
- **B2C (Business-to-Consumer):** Identity management for customer-facing apps (scale to millions of users).

Example: Your company uses Microsoft 365. With Azure Entra ID SSO, employees log in once to their laptop and can access Outlook, Teams, SharePoint, and your company's internal web apps — without logging in again each time.

9.3 Azure Key Vault

Azure Key Vault is a **secure storage service for secrets, keys, and certificates**. Never hard-code passwords or connection strings in your code. Store them in Key Vault instead.

Example: BAD (insecure): `string connStr = "Server=myserver;Password=MyPassword123";` GOOD (with Key Vault): `string connStr = await secretClient.GetSecretAsync("DatabaseConnectionString");` // The actual password never appears in your code!

What Key Vault Stores

- **Secrets:** Passwords, API keys, connection strings, certificates
- **Keys:** Cryptographic keys for encryption/decryption (HSM-protected)
- **Certificates:** SSL/TLS certificates with automatic renewal

9.4 Web API Security Best Practices

Layer	Practice	Implementation
Transport	Always use HTTPS	Enforce HTTPS in Azure App Service settings
Authentication	Use token-based auth (JWT + OAuth 2.0)	ASP.NET Identity + Azure AD
Authorization	Least-privilege access control	[Authorize(Roles='Admin')] attributes
Input Validation	Validate all inputs. Never trust user data.	Model validation, data annotations
Secrets	Never hardcode credentials	Azure Key Vault integration
CORS	Only allow known origins	Whitelist specific origins in CORS policy
Rate Limiting	Prevent abuse/DDoS	Azure API Management throttling policies

9.5 Azure Role-Based Access Control (RBAC)

RBAC lets you grant the exact permissions users need — no more, no less. Assign roles to users, groups, or service principals.

Built-in Role	Permissions
Owner	Full access + can grant access to others
Contributor	Create and manage resources, but cannot grant access
Reader	View resources only. Cannot make changes.
User Access Administrator	Manage user access only.

Example: Your developer needs to deploy to Azure App Service but shouldn't be able to delete databases. Assign them 'Contributor' on the App Service resource group only. They get exactly what they need.

■ Chapter 10: Hosting & Consuming Web APIs on Azure

10.1 Azure App Service for Web APIs

Azure App Service is the recommended platform for hosting ASP.NET Web APIs. Fully managed, auto-scaling, with built-in SSL, monitoring, and deployment tools.

Deployment Options

Method	Description	Best For
Visual Studio Publish	Right-click → Publish → Azure App Service. Guided wizard.	Individual developers, quick deployment
Azure CLI	az webapp up / az webapp deployment source config-zip	Scripting, automation
GitHub Actions / Azure DevOps	Automatic deployment on every git push (CI/CD)	Teams, production environments
Docker Container	Package your API in a Docker container, deploy to App Service or AKS	Microservices, consistent environments
ARM/Bicep Template	Infrastructure as Code — deploy app + infrastructure together	Repeatable, consistent deployments

10.2 Consuming a Web API from .NET Clients

Example: Using HttpClient in C# to consume an API: // Setup (do this once, reuse HttpClient) var client = new HttpClient(); client.BaseAddress = new Uri("https://api.example.com/"); client.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer", token); // GET request var response = await client.GetAsync("api/students/5"); var student = await response.Content.ReadFromJsonAsync(); // POST request var newStudent = new Student { Name = "Ali", Email = "ali@example.com" }; var postResponse = await client.PostAsJsonAsync("api/students", newStudent); // Check result if (postResponse.IsSuccessStatusCode) { Console.WriteLine("Created successfully!"); }

10.3 Azure API Management (APIM)

APIM is a service that puts a **managed gateway in front of your APIs**, providing: authentication, rate limiting, caching, transformation, analytics, and developer portal.

APIM Feature	Purpose
Rate Limiting / Throttling	Limit calls per API key (e.g., 1000 calls/minute)

APIM Feature	Purpose
Authentication	Validate JWT tokens, API keys, OAuth before reaching backend
Caching	Cache API responses to reduce backend load
Transformation	Modify request/response headers, bodies without touching backend code
Developer Portal	Auto-generated documentation site for API consumers
Analytics	Track API usage, errors, latency per endpoint/consumer
Versioning	Support multiple API versions simultaneously

Example: You have 3 different APIs: /users, /orders, /products. APIM sits in front of all three. External developers get API keys from the portal. APIM enforces they can only call 100 times/minute. Unauthorized calls are blocked before reaching your servers.

■ Chapter 11: WCF Services

11.1 What is WCF?

Windows Communication Foundation (WCF) is a framework for building service-oriented applications. WCF services can communicate using multiple protocols (HTTP, TCP, Named Pipes, MSMQ) and message formats.

***Tip:** In modern development, REST APIs (Web API) and gRPC have largely replaced WCF. However, many enterprise systems still use WCF, so understanding it is important for maintaining legacy systems.*

WCF vs. Web API

Feature	WCF	Web API (REST)
Protocols	HTTP, TCP, MSMQ, Named Pipes	HTTP/HTTPS only
Message Format	SOAP (XML)	JSON (usually), XML
Transport Security	WS-Security, message-level security	HTTPS + OAuth
Interoperability	Any SOAP client (.NET, Java, SAP)	Any HTTP client
Complexity	High — complex configuration	Simple — convention-based
Performance	Good (TCP binding very fast)	Better (lighter payload)
Use Case	Enterprise, legacy, needs non-HTTP protocols	Web/mobile apps, microservices

WCF Core Concepts

Concept	Description
Service Contract [ServiceContract]	Interface defining what operations the service exposes
Operation Contract [OperationContract]	Individual methods that clients can call
Data Contract [DataContract]	Classes that define data exchanged between client and service
Binding	How the service communicates (basicHttpBinding, netTcpBinding, wsHttpBinding)
Endpoint	Where the service is accessible: Address + Binding + Contract (ABC)

Example: WCF Service Example: [ServiceContract] public interface ICalculator { [OperationContract] double Add(double a, double b); } public class CalculatorService : ICalculator { public double Add(double a, double b) => a + b; } // Client calls: var client = new CalculatorClient(); double result = client.Add(10.5, 20.3); // Returns 30.8

■ Chapter 12: Azure Serverless & API Management

12.1 Serverless Computing

In **serverless computing**, you write code (functions) and Azure manages all infrastructure: servers, OS, scaling, availability. You pay only when your code runs.

Traditional Server	Serverless (Azure Functions)
Pay 24/7 even when idle	Pay only when code executes
You manage server, OS, patches	Azure manages everything
Manual scaling	Auto-scales to millions of executions
Always on — minimum cost	Cost can be nearly zero for low traffic

Example: Scenario: You need to send welcome emails when a new user registers. With Azure Functions: Write a function that triggers when a new row appears in Cosmos DB → sends email via SendGrid. The function runs in milliseconds, costs fractions of a paisa per execution, and requires zero server management.

Azure Functions Triggers

Trigger Type	Fires When	Example
HTTP Trigger	HTTP request received	REST API endpoint
Timer Trigger	Schedule (cron expression)	Daily report generation at midnight
Blob Trigger	File uploaded to Blob Storage	Resize image when photo uploaded
Queue Trigger	Message added to Azure Queue	Process order from queue
Cosmos DB Trigger	Document changed in Cosmos DB	Update search index when product updated
Event Hub Trigger	Events arrive in Event Hub	Process IoT sensor data in real-time

Example: Timer-triggered Azure Function (runs every day at midnight): `[FunctionName("DailyReport")] public async Task Run( [TimerTrigger("0 0 0 * * *")] TimerInfo timer, ILogger log) { log.LogInformation("Generating daily report..."); await GenerateAndEmailReport(); }`

12.2 Azure Logic Apps

Logic Apps are **low-code/no-code workflow automation** tools. Build workflows visually by connecting 300+ connectors (Office 365, Salesforce, Twitter, SQL, etc.).

Example: Workflow: When a new row is added to an Excel file in SharePoint → extract the data → insert into Azure SQL Database → send confirmation email to manager. All done visually with zero code.

12.3 Azure Event Grid

Event Grid is a **fully managed event routing service**. It reacts to events from Azure services and routes them to subscribers (Azure Functions, Logic Apps, webhooks).

Example: When a user uploads a file to Blob Storage → Event Grid fires a 'BlobCreated' event → routes to an Azure Function → Function processes the file → sends notification to a Queue.

■ Chapter 13: CI/CD with Azure DevOps

13.1 What is CI/CD?

CI (Continuous Integration): Developers frequently merge code into a shared repository. Each merge triggers automated build and tests. Detect integration issues early.

CD (Continuous Delivery): After CI passes, automatically deploy to staging/production environments. Software is always in a deployable state.

Example: Developer pushes code to GitHub. Within 5 minutes: Azure Pipelines builds the app → runs 500 unit tests → if all pass, deploys to staging environment → team reviews → one-click deploy to production. No manual build/deploy steps needed.

CI/CD Benefits

Without CI/CD	With CI/CD
Manual builds — developer runs build script	Automatic build on every commit
Tests run manually (or skipped)	Tests run automatically — can't skip
Deploy manually (error-prone)	Automatic deployment to staging
Integration issues found late (at release)	Integration issues found immediately
Release every few months (big risky releases)	Release multiple times per day (small, safe releases)

13.2 Azure DevOps Components

Component	Purpose
Azure Repos	Git-based source code repository (like GitHub within Azure)
Azure Pipelines	CI/CD automation — build, test, deploy pipelines
Azure Boards	Work item tracking (user stories, bugs, tasks) — like Jira
Azure Test Plans	Manual and automated test management
Azure Artifacts	Package management — npm, NuGet, Maven packages for your team

13.3 Azure Pipeline Stages

- **Stage 1 — Source Control Trigger:** Developer pushes code to Azure Repos or GitHub
- **Stage 2 — Build Stage:** Compile code, resolve dependencies, create build artifacts

- **Stage 3 — Test Stage:** Run unit tests, integration tests, code quality checks (SonarQube)
- **Stage 4 — Package Stage:** Create Docker image or deployment package
- **Stage 5 — Deploy to Staging:** Deploy automatically to staging environment for testing
- **Stage 6 — Manual Approval (optional):** Team lead reviews staging and approves production deployment
- **Stage 7 — Deploy to Production:** Automated deployment to production Azure environment

Example: Azure Pipeline YAML (azure-pipelines.yml): trigger: - main # Trigger on every push to main branch
pool: vmImage: 'windows-latest' steps: - task: DotNetCoreCLI@2 displayName: 'Build' inputs: command: 'build'
projects: '**/*.csproj' - task: DotNetCoreCLI@2 displayName: 'Run Tests' inputs: command: 'test' projects:
'**/*Tests/*.csproj' - task: AzureWebApp@1 displayName: 'Deploy to Azure' inputs: azureSubscription: 'My
Azure Subscription' appName: 'my-web-app'

■ Chapter 14: Traffic Management & Web Service Monitoring

14.1 Azure Traffic Manager

Traffic Manager is a **DNS-based traffic load balancer** that distributes traffic to services across global Azure regions based on routing rules.

Traffic Routing Methods

Routing Method	Description	Use Case
Priority	Send all traffic to primary endpoint. Failover to secondary if primary fails.	Disaster recovery — primary in UK, backup in US
Weighted	Distribute traffic across endpoints by weight (percentage).	A/B testing — 90% to stable version, 10% to new version
Performance	Route to endpoint with lowest latency for the user.	Global apps — route users to closest region
Geographic	Route based on user's geography. Comply with data residency laws.	GDPR — European users must be served from European datacenters
Subnet	Route based on user's IP address range.	Corporate users (VPN IP range) → internal API; others → public API

Example: Your app has servers in US (primary) and Europe (secondary). Traffic Manager with Priority routing: all traffic → US server. If US goes down, Traffic Manager detects failure within 30 seconds and routes all traffic → Europe automatically. Users may see a brief delay but service remains available.

14.2 Azure Load Balancer vs. Application Gateway vs. Traffic Manager

Service	Layer	Scope	Use Case
Load Balancer	Layer 4 (TCP/UDP)	Regional — single Azure region	Distribute traffic across VMs in same region
Application Gateway	Layer 7 (HTTP/HTTPS)	Regional — with WAF, SSL termination, URL routing	Web apps needing advanced HTTP features + security
Traffic Manager	DNS Layer	Global — multiple regions	Global apps deployed in multiple regions

14.3 Azure Front Door

Azure Front Door is a global, scalable entry point for web applications. It combines CDN, load balancing, WAF (Web Application Firewall), and SSL termination in one service.

- Routes users to the fastest available backend globally
- Built-in WAF protects against OWASP Top 10 vulnerabilities
- Caches static content at global edge locations
- Health probes detect failed backends and reroute instantly
- SSL offloading — encrypts at edge, communicates with backend over HTTP

■ Chapter 15: Deploying Web Applications & Azure App Service Apps

15.1 Azure App Service Deployment

Azure App Service is the primary service for hosting web applications, REST APIs, and mobile backends. It supports .NET, .NET Core, Java, Python, Node.js, PHP, Ruby.

Deployment Slots

Deployment slots let you have multiple versions of your app running simultaneously (e.g., production, staging, dev). You can **swap slots** to promote staging to production with zero downtime.

Example: Deploy new version to 'staging' slot. Test thoroughly. When ready, click 'Swap' — staging becomes production instantly. Zero downtime. If something goes wrong, swap back in 30 seconds.

App Service Plans

Plan	Use Case	Scaling	Features
Free / Shared	Dev/test only	None	Limited compute, no custom domain
Basic	Low-traffic production	Manual scale up to 3 instances	Custom domain, SSL
Standard	Production apps	Auto-scale up to 10 instances	Deployment slots, Traffic Manager
Premium	High-performance apps	Auto-scale up to 30 instances	All Standard features + more
Isolated (ASE)	Maximum isolation & security	Auto-scale up to 100	Private VNet, dedicated hardware

15.2 Auto-Scaling

Auto-scaling automatically adjusts the number of instances of your app based on demand. Save money during low traffic, maintain performance during peaks.

Scaling Type	Description	Example
Horizontal Scale-Out	Add more instances of the app	2 VMs → 10 VMs when traffic spikes
Horizontal Scale-In	Remove instances when traffic decreases	10 VMs → 2 VMs at night
Vertical Scale-Up	Move to a more powerful VM size	2 CPU → 8 CPU (change App Service Plan)

Example: Auto-scale rule: When CPU > 70% for 5 minutes → add 2 instances. When CPU < 30% for 10 minutes → remove 1 instance. Minimum: 2 instances. Maximum: 10 instances. Your app handles both quiet nights and busy sale days automatically.

15.3 Docker Containers on Azure

Containerize your application using Docker, then deploy to Azure Container Instances (ACI) for simple scenarios or Azure Kubernetes Service (AKS) for production microservices.

Example: Basic Docker workflow on Azure: # 1. Build Docker image `docker build -t myapp:v1 .` # 2. Push to Azure Container Registry (ACR) `az acr login --name myacr` `docker tag myapp:v1 myacr.azurecr.io/myapp:v1` `docker push myacr.azurecr.io/myapp:v1` # 3. Deploy to Azure Container Instances `az container create --resource-group MyRG --name myapp-container \ --image myacr.azurecr.io/myapp:v1 --dns-name-label myapp \ --ports 80` # 4. App is live at: `myapp.eastus.azurecontainer.io`

15.4 Azure Kubernetes Service (AKS)

AKS is a fully managed Kubernetes service for running containerized applications at scale. It handles the complexity of Kubernetes cluster management.

Key Kubernetes Concepts

Term	Description	Analogy
Pod	Smallest deployable unit. Contains one or more containers.	A single running application instance
Deployment	Manages multiple pods. Ensures desired number of replicas is running.	Blueprint for running your app at scale
Service	Stable network endpoint for accessing pods (load balanced).	The public address for your app
Node	A virtual machine that runs pods.	A single server in your cluster
Cluster	Collection of nodes managed together.	Your entire Kubernetes infrastructure
Namespace	Virtual cluster within a cluster for isolation.	Separate environments in one cluster

15.5 Summary: Choosing the Right Azure Compute Service

Service	Choose When...
Azure App Service	Web apps, REST APIs, .NET/Node.js/Python apps. Simplest PaaS option.

Service	Choose When...
Azure Functions	Event-driven, short-running code. Serverless. Pay-per-execution.
Azure Container Instances	Simple containerized apps. No orchestration needed. Fast startup.
Azure Kubernetes Service (AKS)	Complex microservices. Many containers. Production at scale.
Azure Virtual Machines	Full OS control needed. Lift-and-shift of existing apps. Legacy software.
Azure Batch	Large-scale parallel processing jobs. Scientific computing, rendering, analytics.

■ Quick Reference: Azure Key Terms

Important Azure Terms

Term	Definition
Subscription	A billing container. All Azure resources belong to a subscription.
Resource Group	Logical container for related Azure resources.
Azure Region	Geographic location of Azure datacenters. Example: East US, West Europe, UAE North.
Availability Zone	Physically separate datacenters within a region. High availability through redundancy.
SLA	Service Level Agreement — Microsoft's uptime guarantee (e.g., 99.9%).
CapEx vs OpEx	Capital Expenditure (buy hardware upfront) vs Operating Expenditure (pay monthly for cloud).
IaaS / PaaS / SaaS	Infrastructure / Platform / Software as a Service — cloud service models.
Scalability	Ability to handle growing workload by adding resources.
Elasticity	Automatic scaling based on demand (scale out AND back in).
High Availability	System remains operational even when components fail.
Fault Tolerance	System continues operating despite failures.
Disaster Recovery	Restoring system after a major failure. RTO (Recovery Time) and RPO (Recovery Point).
Load Balancing	Distributing traffic across multiple servers for performance and reliability.
Microservices	Architecture where app is broken into small, independent, deployable services.
API Gateway	Single entry point for all API calls. Handles auth, routing, rate limiting.
CDN	Content Delivery Network — distributes content globally at edge servers.
Blob	Binary Large Object — unstructured data storage in Azure.
Container	Lightweight, isolated package with application and its dependencies.

Term	Definition
Kubernetes (K8s)	Container orchestration platform — manages, scales, and heals containers.
CI/CD	Continuous Integration / Continuous Delivery — automated build, test, deploy pipeline.
JWT	JSON Web Token — compact, self-contained authentication token.
RBAC	Role-Based Access Control — permissions based on roles, not individual users.
VNet	Virtual Network — isolated private network in Azure cloud.
NSG	Network Security Group — firewall rules for Azure VNets.
ORM	Object-Relational Mapper — maps C# objects to database tables (e.g., Entity Framework).
LINQ	Language-Integrated Query — query syntax built into C#.
ACID	Atomicity, Consistency, Isolation, Durability — database transaction properties.

Common Azure CLI Commands Reference

Action	Command
Login	<code>az login</code>
List subscriptions	<code>az account list</code>
Set subscription	<code>az account set --subscription 'My Sub'</code>
Create resource group	<code>az group create --name MyRG --location eastus</code>
Create VM	<code>az vm create --resource-group MyRG --name MyVM --image UbuntuLTS</code>
Create App Service Plan	<code>az appservice plan create --name MyPlan --resource-group MyRG --sku B1</code>
Create Web App	<code>az webapp create --name MyApp --resource-group MyRG --plan MyPlan --runtime 'dotnet:6'</code>
Deploy ZIP	<code>az webapp deployment source config-zip --resource-group MyRG --name MyApp --src app.zip</code>
Create SQL Server	<code>az sql server create --name MySQLSrv --resource-group MyRG --admin-user myadmin</code>
Create SQL Database	<code>az sql db create --resource-group MyRG --server MySQLSrv --name MyDB</code>
List all VMs	<code>az vm list --output table</code>
Start/Stop VM	<code>az vm start/stop --resource-group MyRG --name MyVM</code>
Show blob storage	<code>az storage account list --output table</code>

Create Key Vault

```
az keyvault create --name MyKV --resource-group MyRG --location eastus
```